Stacks-1

(last in first out)

Agenda:
1. Operations of stack
2. Undo/Redo
3. Stack implementation on dynamic array
4. Stack implementation via Linked list
5. Balanced Parenthesis
6. Evaluate Post fix expression

Stack → linear data structure that follows LIFO principle.

Op:
- push(x): Adds an element x to the top
- pop(): Removes and returns the top element
- peek() / top(): Returns the top element without removing it.
- isEmpty(): Checks if the stack has no elements.

Array Implementation ✓ | Linked List Implementation ✓

Application:

① Balanced Parantheses → { [( )] }

↳ push opening brackets onto the stack. When closing bracket appears, it must match the bracket at top of the stack.

```
public boolean isValid(String s){
    Stack <Character> st = new Stack <>();
    for (int i = 0; i < s.length(); i++){
        char ch = s.charAt(i);
        if (ch == '(' || ch == '{' || ch == '[' ){
            st.push(ch);
        }
        else {
            if (st.isEmpty()) return false;
            char top = st.peek();
            if ((ch == ')' && top == '(') ||
                (ch == '}' && top == '{') ||
                (ch == ']' && top == '[')){
                st.pop();
            }
            else { return false; }
        }
    return st.isEmpty();
}
```

② Postfix Evaluation

An arithmetic expression is given by a string array A of size N. Evaluate the value of an arithmetic expression in Reverse Polish Notation. → Postfix

Valid operators are +, -, *, /. Each string may be an integer or an operator.

Postfix → operators written after their operands.

ex: • ["2", "1", "+", "3", "*"] → ((2) + (1)) * (3) = 9

• ["4", "13", "5", "/", "+"] → (4) + ((13)/(5)) = 6

Code: →

```
public int evalRPN(String[] A){
    Stack<Integer> st = new Stack<>();
    for (String s:A){
        if (s.equals("+") || s.equals("-") || s.equals("*")
                || s.equals("/")){
            int t2 = st.pop();
            int t1 = st.pop();
            if (s.equals("+")) st.push(t1+t2);
            else if (s.equals("-")) st.push(t1-t2);
            else if (s.equals("*")) st.push(t1*t2);
            else if (s.equals("/")) st.push(t1/t2);
        }
        else { st.push(Integer.parseInt(s)); }
    }
    return st.pop();
}
}
```

Stacks-2

Agenda:

1. Nearest Smaller & Greater Element of Left
2. Nearest Smaller & Greater Element on Right
3. Largest Rectangle in a Histogram

① Nearest Smaller Element on left:

This problem asks to find the index j for every element at index i such that $A[j] < A[i]$, where $j<i$ and j is the closest possible index to i. If no such element exist, return -1.

```
public int [] nearestSmallerLeft( int[] A){
        int n = A.length;
        int[] ans = new int[n];
        Stack <Integer> st = new Stack.<>();  //Stores indices

    for (int i=0; i<n; i++){
// Pop elements that are greater than or equal to the current element
        while ( !st.isEmpty() && A[st.peek()] >= A[i]){
                st.pop();
        }

        if (st.isEmpty()) {          // If stack is empty, no smaller element exists on the left
            ans[i] = -1; }
        else { ans[i] = st.peek(); }   // The top of the stack is the index of nearest smaller element

        st.push(i);   //Push current index ~~of the nearest smaller element~~ for future comparisons.
    }
    return ans;      // T.C : O(N)
                     S.C : O(N)
}
```

- If the curr. element $A[i]$ is smaller than the element at the top of the stack, the top element can never be the "nearest smaller" for any future elements because $A[i]$ is both smaller and closer to them.
- Therefore, we "pop" larger elements until we find a smaller one or the stack becomes empty.

② Next Greater Element on Right

Q. Given an array A, find the next greater G[i] for every element A[i] in the array. The next greater element for an element A[i] is the first greater element on the right side of A[i] in the array, A. If not exist return -1.

Sol→

```
public class Solution {
    public int [] nextGreater( int [] A ){
        int n = A.length;
        int [] ans = new int [n];
        Stack <Integer> st = new Stack <>();
        for( int i = n-1 ; i >= 0 ; i--) {
            while ( !st.isEmpty() && st.peek() <= A[i]){
                st.pop();
            }
            if ( st.isEmpty() ) ans[i] = -1 ;
            else ans [i] = st.peek();
            st.push (A[i]);
        }
        return ans;
    }
}
```

---

③ Largest Rectangle in Histogram

→ Problem asks for the max. area of rectangle that can be formed within a histogram.

```
public int LargestRectangleArea ( int [] A) {
    int n = A.length ;
    int [] leftSmaller = nearestSmallerLeft (A);
    int [] rightSmaller = nearestSmallerRight (A);

    int maxArea = 0;
    for (int i = 0 ; i < n ; i++){
        int k ;
        if (rightSmaller[i] == -1){ k = n ;}
        else {k = rightSmaller [i];}
        int j = leftSmaller [i];
        int width = k - j - 1;
        int area = A[i] * width;
        maxArea = Math.max (maxArea , area);
    }
    return maxArea;
}
```

Logic: For each bar of height $H = A[i]$, the largest rectangle using that specific bar as the min. height extends to the left until it hits a smaller bar and to the right until it hits a smaller bar.

- Left Boundary (j) : Index of Nearest Smaller Element on Left.
- Right Boundary (k): Index of Nearest Smaller Element on Right.
- Width : $(k - j - 1)$
- Area: $A[i] \times (k - j - 1)$.

T.C : $O(N)$